

HSH-64 设计文档

本文档通俗的语言解释了HSH-64结构，以及该结构的设计。该设计最初的想法为能否使用一串数字编码语义，现在解释的是最终版本。

在之前的版本当中(详见Github)，项目采用了HSH-20，HSH-32的设计，最终迭代为HSH-64，现在让我们开始梳理

1. 词表与两组嵌入

现在假设我们有一个固定词表：

$$\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$$

- $\mathcal{X}$ ：词表集合；
- N ：词表中项目的总数；
- $\mathbf{x}_i$ ：第 i 个项目的学生嵌入。

其中包含 N 个项目（在 HSH-64 的实验中，因为设备受到限制，项目只采用了较小的词表 $N = 3,109$ ）。每个项目 i 对应一个 D 维连续向量：

$$\mathbf{x}_i \in \mathbb{R}^D$$

- $\mathbf{x}_i$ ：第 i 个项目的学生嵌入；
- $\mathbb{R}^D$ ： D 维实数空间；
- D ：学生嵌入的维度（bge-small 中 $D = 512$ ）。

含义： $\mathbf{x}_i$ 是轻量化编码器（最终选用 **bge-small**）吐出的向量，维度 $D = 512$ 。它体积小、速度快，用于线上实际检索时的输入。

此外，我们还有一组「教师」嵌入：

$$\mathcal{T} = \{\mathbf{t}_1, \dots, \mathbf{t}_N\}$$

- $\mathcal{T}$ ：教师嵌入集合；
- $\mathbf{t}_i$ ：第 i 个项目的教师嵌入；
- N ：词表中项目的总数（与学生集合一一对应）。

含义： $\mathbf{t}_i$ 由更强的模型（最终选用 **bge-large**）产生，专门用来提供高精度的「标准答案」，用于在后续训练验证准确度。它只在离线阶段使用。

符号	来源模型	作用	使用阶段
$\mathbf{x}_i$	bge-small	学生向量，轻量化	线上推理
$\mathbf{t}_i$	bge-large	教师向量，高精度	离线训练/评估

2. 教师相似度矩阵

用教师向量计算项目 i 与项目 j 之间的余弦相似度：

$$s_{ij} = \frac{\mathbf{t}_i^\top \mathbf{t}_j}{\|\mathbf{t}_i\| \|\mathbf{t}_j\|}$$

- s_{ij} ：项目 i 与项目 j 之间的余弦相似度；
- $\mathbf{t}_i, \mathbf{t}_j$ ：项目 i 和项目 j 的教师嵌入；
- $\mathbf{t}_i^\top \mathbf{t}_j$ ：两个教师嵌入的点积；
- $\|\mathbf{t}_i\|$ ：向量 $\mathbf{t}_i$ 的 L2 范数（长度）。

为什么用余弦相似度？

因为它只关心两个向量的「方向」是否一致，而忽略长度差异，适合度量语义相似性。

离线作用：在离线阶段，用 bge-large 把词表 $\mathcal{X}$ 中所有词两两之间的余弦相似度都算出来，存储为一个 $N \times N$ 的相似度矩阵 S ：

$$S = \begin{bmatrix} s_{11} & s_{12} & \cdots & s_{1N} \\ s_{21} & s_{22} & \cdots & s_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ s_{N1} & s_{N2} & \cdots & s_{NN} \end{bmatrix}$$

- S ： $N \times N$ 的教师余弦相似度矩阵；
- s_{ij} ：矩阵第 i 行第 j 列的元素，即项目 i 与项目 j 的余弦相似度；
- N ：词表大小。

矩阵 S 后续有两个重要作用：

1. 定义每个项目的「真实 top- K 邻居」；
2. 在训练阶段为学生模型提供监督信号（例如蒸馏损失）。w

3. 真实 top- K 邻居集合（标准答案）

在这里，我们定义标准答案的集合

项目 i 的真实 top- K 邻居集合记为：

$$\mathcal{G}_i^{(K)} = \arg \max_{\mathcal{J} \subseteq [N] \setminus \{i\}, |\mathcal{J}|=K} \sum_{j \in \mathcal{J}} s_{ij}$$

- $\mathcal{G}_i^{(K)}$ ：项目 i 的真实 top- K 邻居集合（标准答案）；
- $[N]$ ：词表索引集合 $\{1, 2, \dots, N\}$ ；
- $\mathcal{J}$ ：从 $[N] \setminus \{i\}$ 中选出的包含 K 个索引的子集；
- $|\mathcal{J}|$ ：集合 $\mathcal{J}$ 的元素个数；
- K ：邻居集合的大小；
- s_{ij} ：项目 i 与项目 j 的余弦相似度；
- $\arg \max$ ：取使后面表达式最大的那个 $\mathcal{J}$ 。

3.1 符号拆解

- $[N] = \{1, 2, \dots, N\}$ ：词表中所有词的编号。
- $[N] \setminus \{i\}$ ：把查询词 i 自己排除掉——一个词不能算自己的邻居。
- $\mathcal{J} \subseteq [N] \setminus \{i\}$ ：从剩下的 $N - 1$ 个词中挑出一个子集（候选组合）。
- $|\mathcal{J}| = K$ ：这个子集必须恰好包含 K 个词。
- $\sum_{j \in \mathcal{J}} s_{ij}$ ：把这 K 个词分别与 i 的相似度加起来，得到「总分」。
- $\arg \max$ ：找出让总分最大的那个组合。

3.2 直观理解

其实它做的事情很简单：

把与项目 i 最相似的 K 个词挑出来。

因为所有相似度 s_{ij} 都是非负的（余弦相似度在 $[-1, 1]$ 之间，但语义相关的词通常为正），所以让总和最大的组合，就是按 s_{ij} 从高到低排序后取前 K 个。

3.3 举例：「苹果」的 top-10 邻居

假设词表里还有「水果」「红富士」「香蕉」「手机」「汽车」等词，bge-large 算出它们与「苹果」的相似度如下：

候选词	与「苹果」的相似度 s_{ij}
-----	--------------------

水果	0.95
红富士	0.90
香蕉	0.85
手机	0.30
汽车	0.10
...	...

如果 $K = 10$ ，那么 $\mathcal{G}_{\text{苹果}}^{(10)}$ 就是相似度最高的 10 个词，例如：

$$\mathcal{G}_{\text{苹果}}^{(10)} = \{\text{水果, 红富士, 香蕉, ...}\}$$

此时，集合 $\mathcal{G}_{\text{苹果}}^{(10)}$ 当中拥有 10 个元素，而这就是项目「苹果」的**标准答案集合**。

4. 哈希检索与 Recall@K

语义哈希函数 h 把轻量学生向量 $\mathbf{x}_i$ 映射为一个二进制码：

$$h : \mathbb{R}^D \rightarrow \{0, 1\}^M$$

- h ：语义哈希函数；
- $\mathbb{R}^D$ ： D 维连续嵌入空间（学生嵌入所在空间）；
- $\{0, 1\}^M$ ： M 维二进制码空间；
- M ：二进制码长度（HSH-64 中 $M = 52$ ）。

在 HSH-64 中， $M = 52$ 是语义相似码长度，最终包装在 64 位整数中。

检索函数返回与查询码 Hamming 距离最小的 top- K 个项目：

$$\mathcal{R}_i^{(K)} = \arg \min_{\mathcal{J} \subseteq [N] \setminus \{i\}, |\mathcal{J}|=K} \sum_{j \in \mathcal{J}} d_H(h(\mathbf{x}_i), h(\mathbf{x}_j))$$

- $\mathcal{R}_i^{(K)}$ ：HSH-64 系统为项目 i 返回的 top- K 结果集合；
- $\mathcal{J}$ ：候选的 top- K 索引子集；
- K ：返回结果的数量；
- $d_H(\cdot, \cdot)$ ：Hamming 距离函数；
- $h(\mathbf{x}_i)$ ：项目 i 的哈希码；
- $h(\mathbf{x}_j)$ ：项目 j 的哈希码；

- 其中 i 是被查询的那个词, j 是被拿来和 i 比较的其他词
- $\arg \min$: 取使总 Hamming 距离最小的那个 $\mathcal{J}$, 也就是最相似的。

含义: $\mathcal{R}_i^{(K)}$ 是 HSH-64 系统实际返回的结果集合。

4.1 Recall@K 定义

整个系统的优化目标是最大化 Recall@K:

$$\text{Recall@K} = \frac{1}{N} \sum_{i=1}^N \frac{|\mathcal{R}_i^{(K)} \cap \mathcal{G}_i^{(K)}|}{K}$$

- Recall@K: 前 K 个结果的平均召回率;
- N : 词表中项目的总数;
- $\mathcal{R}_i^{(K)}$: HSH系统返回的 top- K 结果集合;
- $\mathcal{G}_i^{(K)}$: 教师给出的真实 top- K 邻居集合;
- $\cap$: 两个集合的交集;
- $|\cdot|$: 集合的元素个数;
- K : 返回结果的数量。

4.2 符号拆解

- $\mathcal{R}_i^{(K)}$: 哈希模型返回的 top- K 结果。
- $\mathcal{G}_i^{(K)}$: bge-large 给出的标准答案 top- K 。
- $\mathcal{R}_i^{(K)} \cap \mathcal{G}_i^{(K)}$: 两个集合的交集, 即「命中正确答案」的项目。
- $|\cdot|$: 交集中元素个数。
- $\frac{|\cap|}{K}$: 把命中数转化为比例。例如 $K = 10$, 命中 7 个, 则该查询的 $\text{Recall@10} = 0.7$ 。
- $\frac{1}{N} \sum_{i=1}^N$: 对所有 N 个查询词取平均, 避免只挑表现好的词来算。

4.3 直观理解

Recall@K 回答的问题是:

系统返回的前 K 个结果中, 有多少确实是语义上最相关的词?

取值范围 $[0, 1]$, 越接近 1 越好。它是 HSH-64 最核心的评价指标。

5. 小结

到这里，我们已经把问题完整定义了一遍：

- 1. **输入**： N 个词的词表，以及每个词的轻量学生向量 $\mathbf{x}_i$ 和强教师向量 $\mathbf{t}_i$ 。
- 2. **标准答案**：用教师向量算余弦相似度，为每个词定义真实 top- K 邻居集合 $\mathcal{G}_i^{(K)}$ 。
- 3. **模型输出**：哈希函数 h 把学生向量映射为二进制码，按 Hamming 距离返回 top- K 集合 $\mathcal{R}_i^{(K)}$ 。
- 4. **优化目标**：最大化 Recall@ K ，即让 $\mathcal{R}_i^{(K)}$ 与 $\mathcal{G}_i^{(K)}$ 尽可能重合。

接下来就可以进入 HSH-64 的编码结构、训练流程和检索算法。

6. HSH-64 编码结构

HSH-64 把整个语义信息压缩进一个 64 位无符号整数：

$$c = \underbrace{(f_3 f_2 f_1 f_0)}_{\text{feat:4}} \parallel \underbrace{(s_{51} \cdots s_1 s_0)}_{\text{sim:52}} \parallel \underbrace{(a_7 \cdots a_1 a_0)}_{\text{abs:8}}$$

- c ：一个 HSH-64 编码（64 位无符号整数）；
- $f_3 f_2 f_1 f_0$ ：feat 字段的 4 个比特；
- $s_{51} \cdots s_1 s_0$ ：sim 字段的 52 个比特；
- $a_7 \cdots a_1 a_0$ ：abs 字段的 8 个比特；
- $\parallel$ ：比特拼接操作符。

三个字段各司其职：

字段	位数	作用	例子/容量
feat	4	词性/粗粒度类别标签	$2^4 = 16$ 个粗类
sim	52	核心语义相似度编码	$2^{52} \approx 4.5 \times 10^{15}$ 种状态
abs	8	簇内完美哈希标识	$2^8 = 256$ 个唯一 ID
总计	64	一个 u64 ，一次 POPCNT	每项仅需 8 字节

两个码 c 与 c' 的距离用一次硬件 popcount 完成：

$$d_H(c, c') = \text{popcnt}(c \oplus c')$$

- $d_H(c, c')$ ：码 c 与码 c' 之间的 Hamming 距离；
- $\text{popcnt}(\cdot)$ ：CPU popcount 指令，统计二进制中 1 的个数；

- $\oplus$: 按位异或运算;
- c, c' : 两个待比较的 HSH-64 编码。

6.1 abs 字段：平局决胜机制

abs 不是语义信号，而是**平局决胜机制** (tie-breaker)。

想象两个词共享相同的 (feat, sim) 前缀，只在 abs 上不同：它们最多相差 8 位，仍处在紧密的 Hamming 邻域内。而 sim 不同的两个词通常相差更多位。因此：

- abs 决定簇内排序；
- abs 不主导跨簇语义距离；
- 也可以把 abs 当作硬过滤器：要求 (feat, sim) 完全匹配后再看 abs，实现桶内完美哈希查找。

7. Deep Hash MLP 架构

HSH-64 用一个极小的 MLP 把 bge-small 的 512 维向量映射成 52 位语义码。

7.1 前向流程

1. 中心化：输入减去训练集均值

$$\mathbf{x}' = \mathbf{x} - \boldsymbol{\mu}$$

- $\mathbf{x}$: 原始学生嵌入；
- $\boldsymbol{\mu}$: 训练集上学生嵌入的均值向量；
- $\mathbf{x}'$: 中心化后的学生嵌入。

2. MLP 投影：单隐藏层 ReLU

$$\mathbf{u} = \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x}' + \mathbf{h}_1) + \mathbf{h}_2$$

- $\mathbf{u}$: 连续 logits 向量；
- $\mathbf{W}_1$: 输入层到隐藏层的权重矩阵；
- $\mathbf{h}_1$: 隐藏层偏置；
- σ : ReLU 激活函数，全称 Rectified Linear Unit (修正线性单元)；
- $\mathbf{W}_2$: 隐藏层到输出层的权重矩阵；
- $\mathbf{h}_2$: 输出层偏置。

- 隐藏层先做一个线性变换 $\mathbf{W}_1 \mathbf{x}' + \mathbf{h}_1$ ，再经过 ReLU。它的作用是：
 - 引入非线性：如果没有激活函数，无论堆多少层，神经网络都等价于一个线性变换，表达能力很弱；
 - 计算简单：只需要判断正负，没有复杂的指数/三角运算；
 - 缓解梯度消失：正数区域梯度恒为 1，反向传播时梯度不容易衰减。
 - 简单说，ReLU 就是「把负数变成 0，正数保持不变」。
- 这里有个理解的误区，隐藏层包括 $\sigma(\mathbf{W}_1 \mathbf{x}' + \mathbf{h}_1)$ ，在这里 σ 只对隐藏层生效，而权重 $\mathbf{W}_2$ ，以及偏置 $\mathbf{h}_2$ 并不受限制，也就是说，最后 $\mathbf{u}$ 的取值范围可以是正数或者负数，并不影响后续符号量化的输入

3. 符号量化：把连续 logit 变成二进制码

$$\mathbf{b} = \frac{1 + \text{sign}(\mathbf{u})}{2} \in \{0, 1\}^{52}$$

- $\mathbf{b}$ ：量化后的 52 位二进制码；
- $\text{sign}(\mathbf{u})$ ：符号函数，正数输出 +1，负数输出 -1；
- $\{0, 1\}^{52}$ ：52 维二进制向量空间。

其中 $\mathbf{u} \in \mathbb{R}^{52}$ 叫 logits， $\mathbf{b} \in \{0, 1\}^{52}$ 是最终 sim 码。

7.2 输出层可替换

MLP 的隐藏层共享，但输出维度 O 随阶段变化：

- O ：MLP 输出层的维度；
- **阶段 1**（教师蒸馏）： $O = 1024$ ，匹配 bge-large 的输出维度；
- **阶段 2**（哈希训练）： $O = 52$ ，生成最终语义码。

阶段 2 直接复用阶段 1 训练好的隐藏层权重，只替换最后一层，从而利用阶段 1 学到的连续投影初始化。

7.3 直通估计器（STE）

由于符号函数 $\text{sign}(\cdot)$ 几乎处处梯度为零，无法直接反向传播。STE 的做法是：

- **前向传播**：用真实的符号量化；
- **反向传播**：把梯度近似为恒等映射。

$$\frac{\partial \mathbf{b}}{\partial \mathbf{u}} \approx \mathbf{I}$$

- $\frac{\partial \mathbf{b}}{\partial \mathbf{u}}$ ：二进制码 $\mathbf{b}$ 对 logits $\mathbf{u}$ 的雅可比矩阵；

- **I**: 单位矩阵;
- 含义: 反向传播时把符号函数的梯度近似为恒等映射。

补充: 雅可比矩阵定义: 如果有一个函数 $\mathbf{b}=\mathbf{f}(\mathbf{u})$, 输入是向量 $\mathbf{u}$, 输出也是向量 $\mathbf{b}$, 那么雅可比矩阵就是把所有「输出对输入的偏导数」排成一个矩阵:

$$\frac{\partial \mathbf{b}}{\partial \mathbf{u}} = \begin{bmatrix} \frac{\partial b_1}{\partial u_1} & \frac{\partial b_1}{\partial u_2} & \dots & \frac{\partial b_1}{\partial u_{52}} \\ \frac{\partial b_2}{\partial u_1} & \frac{\partial b_2}{\partial u_2} & \dots & \frac{\partial b_2}{\partial u_{52}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial b_{52}}{\partial u_1} & \frac{\partial b_{52}}{\partial u_2} & \dots & \frac{\partial b_{52}}{\partial u_{52}} \end{bmatrix}$$

在这里:

$$\mathbf{b} = \frac{1 + \text{sign}(\mathbf{u})}{2}$$

对于每一个 $\mathbf{b}_j$ 都只依赖于对应的 $\mathbf{u}_j$:

当 $\mathbf{u}_j < 0$ 时, $\mathbf{b}_j = 0$, 而当 $\mathbf{u}_j > 0$ 时, $\mathbf{b}_j = 1$

所以在理论上:

$$\frac{\partial \mathbf{b}_j}{\partial \mathbf{u}_j} = \mathbf{0}$$

这意味着真实的雅可比矩阵几乎处处都是 0 矩阵

但是在这里, *STE* 将它近似为了一个单位矩阵, 因为零矩阵无法反向传播梯度

所以单位矩阵 **I** 的含义为: 对于每一个 $\mathbf{b}_j$ 都只受对应的 $\mathbf{u}_j$ 影响, 且影响程度为 1, 也就是:

$$\frac{\partial \mathbf{b}_j}{\partial \mathbf{u}_k} \approx \mathbf{0} (j \neq k), \frac{\partial \mathbf{b}_j}{\partial \mathbf{u}_j} \approx \mathbf{1}$$

这样, 损失对 logits 的梯度可以直接回传到 MLP 权重, 实现端到端训练。

8. 三阶段训练流程

HSH-64 的训练分为三个阶段, 逐步把「连续教师知识」蒸馏成「高质量二进制码」。

8.1 阶段 1: 连续预训练

目标: 让 MLP 的输出尽量接近 bge-large 的教师向量。

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(\mathbf{x}_i) - \mathbf{t}_i\|_2^2$$

- $\mathcal{L}_{\text{MSE}}$: 均方误差损失;

- N : 词表大小;
- $f_{\theta}(\cdot)$: 参数为 θ 的 MLP 函数;
- $\mathbf{x}_i$: 第 i 个项目的学生嵌入;
- $\mathbf{t}_i$: 第 i 个项目的教师嵌入;
- $\|\cdot\|_2$: L2 范数, 也就是向量的模长, 在这里计算的是内部元素的直线距离 (欧氏距离)。

bge-large 的 1024 维向量包含了丰富的语义结构。先用 MSE 蒸馏, 相当于让学生模型学会一个高质量的连续投影, 为后续二值化打下好基础。

8.2 阶段 2: 离散精调

把输出层换成 52 个超平面, 用 STE 端到端训练。总损失由四项组成:

$$\mathcal{L} = \mathcal{L}_{\text{info}} + \lambda_q \mathcal{L}_{\text{quant}} + \lambda_b \mathcal{L}_{\text{balance}} + \lambda_p \mathcal{L}_{\text{pair}}$$

- $\mathcal{L}$: 阶段 2 的总损失;
- $\mathcal{L}_{\text{info}}$: InfoNCE 损失;
- $\mathcal{L}_{\text{quant}}$: 量化损失;
- $\mathcal{L}_{\text{balance}}$: 平衡损失;
- $\mathcal{L}_{\text{pair}}$: 成对损失;
- $\lambda_q, \lambda_b, \lambda_p$: 对应损失的权重超参数

8.2.1 InfoNCE 损失

$$\mathcal{L}_{\text{info}} = -\frac{1}{|\mathcal{P}^+|} \sum_{(i,j) \in \mathcal{P}^+} \log \frac{e^{\mathbf{u}_i^\top \mathbf{u}_j / \tau}}{\sum_k e^{\mathbf{u}_i^\top \mathbf{u}_k / \tau}}$$

- $\mathcal{L}_{\text{info}}$: InfoNCE 损失;
- $\mathcal{P}^+$: 正样本对集合 (教师 top- P 邻居);
- $|\mathcal{P}^+|$: 正样本对的数量;
- (i, j) : 一个正样本对;
- $\mathbf{u}_i, \mathbf{u}_j$: 项目 i 和项目 j 的连续 logits;
- τ : 温度超参数;
- k : 分母中遍历全部 N 个词表项的索引。

作用: 让语义相似的词在 logits 空间中更近, 不相似的词更远。

8.2.2 量化损失

$$\mathcal{L}_{\text{quant}} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{u}_i - \mathbf{b}_i\|_1$$

- $\mathcal{L}_{\text{quant}}$: 量化损失;
- N : 词表大小;
- $\mathbf{u}_i$: 项目 i 的连续 logits;
- $\mathbf{b}_i$: 项目 i 量化后的二进制码;
- $\|\cdot\|_1$: L1 范数 (绝对值之和)。

作用: 鼓励 logits 尽量接近 ± 1 , 这样量化成二进制码时信息损失小。

8.2.3 平衡损失

$$\mathcal{L}_{\text{balance}} = \frac{1}{M} \sum_{j=1}^M (\bar{b}_j - 0.5)^2$$

- $\mathcal{L}_{\text{balance}}$: 平衡损失;
- M : 二进制码长度 ($M = 52$);
- j : 码的第 j 个比特位;
- $\bar{b}_j$: 第 j 个比特在所有项目上的经验均值;
- 0.5: 理想平衡值 (一半 0、一半 1)。

作用: 防止比特坍塌。理想情况下, 每个比特在全部项目上应该大约一半 0、一半 1, 才能最大化信息容量。

8.2.4 成对损失

$$\mathcal{L}_{\text{pair}} = \frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} (\mathbf{u}_i^\top \mathbf{u}_j - \mathbf{t}_i^\top \mathbf{t}_j)^2$$

- $\mathcal{L}_{\text{pair}}$: 成对损失;
- $\mathcal{P}$: 采样的样本对集合;
- $|\mathcal{P}|$: 样本对的数量;
- $\mathbf{u}_i, \mathbf{u}_j$: 学生 logits (L2 归一化后);
- $\mathbf{t}_i, \mathbf{t}_j$: 教师嵌入 (L2 归一化后);
- $\mathbf{u}_i^\top \mathbf{u}_j$: 学生 logits 的内积;
- $\mathbf{t}_i^\top \mathbf{t}_j$: 教师嵌入的内积。

注意: 计算前 $\mathbf{u}_i$ 与 $\mathbf{t}_i$ 都会先经 L2 归一化, 使内积等价于余弦相似度。

作用: 让学生 logits 之间的相似度结构直接对齐教师嵌入。

8.3 阶段 3: 召回导向的后处理

MLP 训练完成后, 固定模型权重, 直接优化二进制码本身, 目标是提升 Recall@K。

8.3.1 正样本与负样本

对每个查询 i :

- **正样本集合** $\mathcal{P}_i^+$: 教师 top- P 邻居的索引;
- **负样本集合** $\mathcal{P}_i^-$: 当前 Hamming top- K 中、但不在正样本集合里的项目。

8.3.2 样本对权重

$$W_{ij} = \begin{cases} +w_{\text{pos}}, & j \in \mathcal{P}_i^+ \\ -w_{\text{neg}}, & j \in \mathcal{P}_i^- \\ 0, & \text{其他} \end{cases}$$

- W_{ij} : 项目 i 与项目 j 之间的召回导向权重;
- w_{pos} : 正样本权重 (超参数);
- w_{neg} : 负样本权重 (超参数);
- $\mathcal{P}_i^+$: 项目 i 的正样本集合 (教师 top- P 邻居);
- $\mathcal{P}_i^-$: 项目 i 的负样本集合 (Hamming top- K 中的非正样本)。

8.3.3 目标函数

$$\mathcal{O}(C) = \sum_{i,j} W_{ij} \cdot d_H(\mathbf{c}_i, \mathbf{c}_j)$$

- $\mathcal{O}(C)$: 召回导向目标函数, 输入为整个码矩阵 C ;
- C : $N \times M$ 的二进制码矩阵;
- W_{ij} : 项目对 (i, j) 的召回导向权重;
- $d_H(\mathbf{c}_i, \mathbf{c}_j)$: 项目 i 与项目 j 的 Hamming 距离;
- $\mathbf{c}_i, \mathbf{c}_j$: 项目 i 和项目 j 的二进制码。

直观解释:

- 对于正样本 ($W_{ij} > 0$): Hamming 距离越小, 目标函数越小 $\rightarrow$ 拉近它们;
- 对于负样本 ($W_{ij} < 0$): Hamming 距离越大, 目标函数越小 $\rightarrow$ 推远它们。

8.3.4 贪心比特翻转

算法迭代尝试翻转每个码的每一位, 选择能让 $\mathcal{O}(C)$ 下降最多的那个翻转, 直到没有改进为止。这个过程收敛到 1-optimal 状态, 即不存在任何单比特翻转能进一步减小目标函数。

9. 非对称距离评分

符号量化丢弃了连续 logit 的幅度信息。为了缓解这一点，HSH-64 在重排时使用**非对称评分**：

$$s_{\text{asym}}(\mathbf{u}_q, \mathbf{b}_d) = \sum_{j=1}^M u_{q,j} \cdot (2b_{d,j} - 1)$$

- s_{asym} ：非对称相似度评分；
- $\mathbf{u}_q$ ：查询项目的连续 logits（在线计算）；
- $\mathbf{b}_d$ ：数据库项目的二进制码（已预计算）；
- $u_{q,j}$ ：查询 logits 的第 j 维；
- $b_{d,j}$ ：数据库码的第 j 个比特；
- M ：码长（ $M = 52$ ）。

这样，查询投影中置信度高的比特会获得更大权重，比纯 Hamming 距离提供更细粒度的排序。在线代价只是对每个候选做一次点积。

10. 自适应多索引哈希（MIH）

为了在线检索足够快，HSH-64 使用 MIH 把 52 位 sim 码切分成多段，并为每段建倒排索引。

10.1 分段与索引

- 52 位 sim 码切分为 $B = 13$ 段；
- 每段 4 位，共有 $2^4 = 16$ 种可能的段码；
- 每段对应一个倒排索引，记录哪些项目在该段上取这个段码。

10.2 查询过程

给定查询码 $\mathbf{q}$ ：

- $\mathbf{q}$ ：查询项目的 HSH-64 编码；
- $\mathbf{q}_b$ ：查询码的第 b 段（ $b = 1, \dots, B$ ）；
- r ：当前总 Hamming 半径；
- B ：段的总数（ $B = 13$ ）；
- $\lfloor r/B \rfloor$ ：每段允许的最大 Hamming 半径（向下取整）。

查询步骤：

1. 对每一段 $\mathbf{q}_b$ ，枚举 Hamming 半径 $\lfloor r/B \rfloor$ 内的所有段码；
2. 查倒排索引，收集候选项目 ID；

3. 合并所有段的候选列表并去重。

10.3 自适应半径扩展

从 $r = 0$ 开始逐步扩大半径，直到满足：

- r ：当前总 Hamming 半径；
- K ：最终需要的 top- K 结果数；
- α ：粗排因子（默认 $\alpha = 10$ ）；
- $\max(K, \alpha K)$ ：候选池最小目标大小；
- δ ：饱和阈值（例如 10%），控制半径扩展何时停止。

停止条件：

- 候选池大小至少为 $\max(K, \alpha K)$ （保证召回）；
- 新增候选比例低于阈值 δ （说明继续扩大半径收益很小）。

默认粗排因子 $\alpha = 10$ ，即先召回约 100 个候选，再精排取 top-10。

11. 两阶段检索架构

HSH-64 的在线检索分为两个阶段：

11.1 粗排（必须）

1. 查询词经 bge-small 编码为 512 维向量；
2. 小 MLP 投影生成 HSH-64 编码；
3. MIH 快速召回候选池（约 187 项）。

在线栈体积非常小：

- bge-small：约 600 MB（HuggingFace 格式）或更小量化版本；
- MLP 投影器：最小 585 KB；
- 码本：25 KB；
- MIH 索引：81 KB。

11.2 精排（可选）

对粗排召回的少量候选，用 bge-large 计算精确余弦相似度并重新排序。因为候选池很小，这一步代价可控。

- 仅 MIH 粗排：延迟约 0.54 ms；
- MIH + bge-large 精排：延迟约 3.21 ms；
- bge-large 精排可以省略，适合边缘设备部署。

12. 模型集成

为了在不增加在线延迟的情况下进一步提升召回，HSH-64 用不同随机种子和隐藏维度训练多个 Deep Hash 模型。检索时合并各模型的候选集，再用以下策略融合：

- **频次**：被更多模型召回的候选排前面；
- **平均排名**：在各模型中平均排名更高的候选排前面；
- **最小 Hamming 距离**：任一模型中距离最小的候选排前面。

实验表明，四个模型的小规模集成相比单模型能提升约 2-3 个百分点的 Recall@10。

13. 实验结果概览

在 3,109 词中文词表上的主要结果：

配置	Recall@10	说明
h256 单模型	0.7382	轻量化版本，MLP 仅 585 KB
h512 单模型	0.7404	最佳单模型，MLP 1.17 MB
四模型集成	0.7724	候选集合并 + 重排
MIH 粗排 + bge-large 精排（四模型集成）	0.8970	完整两阶段系统

关键结论：

- 52 位语义码已经能保留较强的语义信号；
- 后处理和集成可以显著提升召回；
- 两阶段架构把 bge-large 的调用从 3,109 次降到约 187 次，兼顾速度与精度。

14. 全文总结

HSH-64 解决的核心问题是：

如何把高维语义向量压缩成极短的二进制码，同时保持高召回率

通过这套设计，HSH-64 在 3,109 词中文词表上取得了 0.8970 的 Recall@10，同时保持极低的存储和计算开销

到这里整个 HSH-64 系统已经阐述完毕，文档内容基于更新日志内容进行编辑